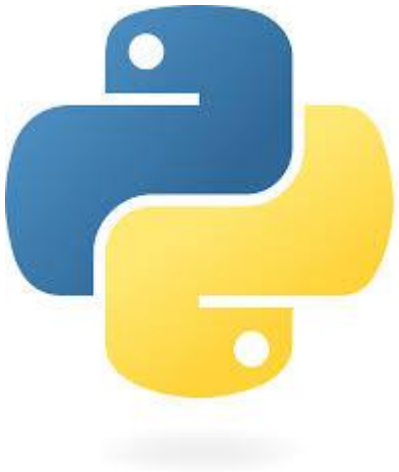
Python



PROGRAMMING FOR EVERYONE

Eng. Omar Shaqra

GroupBy & Aggregation

groupby() splits the data into groups based on one or more columns, then you apply an aggregation function (sum, mean, count, etc.) to each group.

```
# Average grade per subject
df.groupby('Subject')['Grade'].mean()

# Number of students per city
df.groupby('City').size()

# Count of passed/failed per subject
df.groupby(['Subject', 'Passed']).size()
```

Multiple aggregations with agg()

```
# Multiple stats for Grade per Subject
df.groupby('Subject')['Grade'].agg(['mean', 'median', 'min', 'max', 'count', 'std'])

# Different stats for different columns
df.groupby('City').agg({
    'Grade': ['mean', 'max'],
    'Age': 'mean',
    'Student ID': 'count'
})
```

GroupBy & Aggregation

Named aggregation (clean & readable)

```
df.groupby('Subject').agg(
    Avg_Grade = ('Grade', 'mean'),
    Highest_Grade = ('Grade', 'max'),
    Num_Students = ('Student ID', 'count'),
    Pass_Rate = ('Passed', 'mean')
)
```

GroupBy Operations

```
# Count unique values
df.groupby('City')['Subject'].nunique()

# First / last row in each group
df.groupby('City').first()
df.groupby('City').last()

# Transform → same shape as original, group-wise calculation
df['Avg_Grade_in_City'] = df.groupby('City')['Grade'].transform('mean')

# Filter groups (keep only groups that meet condition)
df.groupby('City').filter(lambda x: x['Grade'].mean() > 75)
```

GroupBy & Aggregation

Combining with sorting

```
best_students = df.loc[
    df.groupby('Subject')['Grade'].idxmax(skipna=True).dropna()
]

print(best_students[['Name', 'Subject', 'Grade']])
```

Merging, Joining & Concatenating DFs

Why combine DataFrames?

- Almost every real project involves data from multiple sources/tables that need to be brought together.
- Pandas gives you three main tools:
  - •pd.concat() → stacking DataFrames (vertically or horizontally)
  - •df.merge() → SQL-like join (most powerful & commonly used)
  - •df.join() → join on index (simpler when index is the key)

```python
# Example: two small DataFrames
df1 = pd.DataFrame({
    'Name': ['Ahmed', 'Sara', 'Omar'],
    'Grade': [88, 92, 65]
})

df2 = pd.DataFrame({
    'Name': ['Lina', 'Khaled'],
    'Grade': [78, 55]
})

# Concat vertically (append rows) - default
pd.concat([df1, df2], ignore_index=True)

# Concat horizontally (side by side)
pd.concat([df1, df2], axis=1)
```

Merging, Joining & Concatenating DFs

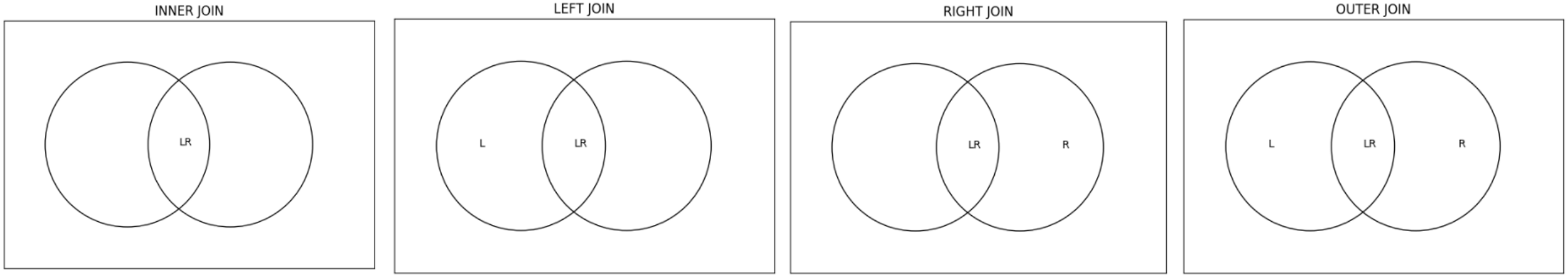
many files

```
files = ['jan.csv', 'feb.csv', 'mar.csv']
dfs = [pd.read_csv(f) for f in files]

all_data = pd.concat(dfs, ignore_index=True)

print("after concat :")
print(all_data)
print()

print("num of rows :", len(all_data))
```

Merging, Joining & Concatenating DFs

df.merge()

- The most powerful (SQL JOIN)



Merging, Joining & Concatenating DFs

```
# Inner join (default) - only matching IDs
print("Inner join")
print(pd.merge(students, scores, on='Student_ID', how='inner'))

# Left join - keep all from left (students), NaN where no match
print("Left join")
print(pd.merge(students, scores, on='Student_ID', how='left'))

# Right join - keep all from right
print("Right join")
print(pd.merge(students, scores, on='Student_ID', how='right'))
# Outer join - keep all from both
print("Outer join")
print(pd.merge(students, scores, on='Student_ID', how='outer'))
```

Merging, Joining & Concatenating DFs

```
# Different column names
pd.merge(students, scores, left_on='Student_ID', right_on='ID')

# Suffixes when columns have same name
pd.merge(students, scores, on='Student_ID', suffixes=('_student', '_score'))

# Merge on index
print(pd.merge(students.set_index('Student_ID'),
               scores.set_index('Student_ID'),
               left_index=True, right_index=True))
```

```
students.set_index('Student_ID').join(scores.set_index('Student_ID'), how='left')
```

Pivot Tables & Crosstab

Why use pivot tables in Pandas?

- Quick way to summarize large datasets
- Create cross-tabulations (contingency tables)
- Group, aggregate, and reshape data in one step
- Very similar to Excel / Google Sheets pivot tables

```
# Average grade per subject
pd.pivot_table(df, values='Grade', index='Subject', aggfunc='mean')

# Multiple aggregations
pd.pivot_table(df,
               values='Grade',
               index='Subject',
               aggfunc=['mean', 'max', 'min', 'count'])
```

Pivot Tables

With rows and columns (2D pivot)

```
# Average grade by Subject (rows) and City (columns)
pd.pivot_table(df,
               values='Grade',
               index='Subject',
               columns='City',
               aggfunc='mean',
               fill_value=0)   # replace NaN with 0

# Add totals (margins)
pd.pivot_table(df,
               values='Grade',
               index='Subject',
               columns='City',
               aggfunc='mean',
               margins=True,          # adds All row & column
               margins_name='Total')
```

Matplotlib

Matplotlib – The Foundation of Plotting in Python

What is Matplotlib?

- The most widely used plotting library in Python
- Extremely flexible and customizable
- Inspired by MATLAB plotting syntax
- Powers many higher-level tools:
  - Pandas .plot()
  - Seaborn
  - Parts of Plotly, Bokeh, etc.

Two Main Ways to Use Matplotlib

Pyplot style

- (MATLAB-like – quick & simple)

```
import matplotlib.pyplot as plt

x = [0, 1, 2, 3, 4]
y = [10, 20, 25, 30, 15]

plt.plot(x, y)
plt.title("Quick Plot Example")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
plt.grid(True)
plt.show()
```

Object-Oriented style

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots()              # fig = whole figure, ax = the plot area
ax.plot(x, y, color='teal', marker='o', linestyle='--')
ax.set_title("Object-Oriented Style")
ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.grid(True)
plt.show()
```

Matplotlib

Common Matplotlib Functions

Function / Method	What it does (English explanation)	Example Usage
`plt.plot()`	Draws a line plot connecting data points (default plot type)	`plt.plot(x, y)`
`plt.scatter()`	Draws individual points (good for showing relationships or clusters)	`plt.scatter(x, y, color='red', s=100)`
`plt.bar()`	Draws vertical bars (for comparing categories)	`plt.bar(categories, values)`
`plt.barh()`	Draws horizontal bars (useful when category names are long)	`plt.barh(categories, values)`
`plt.hist()`	Draws a histogram (shows distribution of one numeric variable)	`plt.hist(data, bins=20)`
`plt.boxplot()`	Draws a box plot (shows median, quartiles, outliers)	`plt.boxplot(scores)`
`plt.pie()`	Draws a pie chart (shows proportions of a whole)	`plt.pie(sizes, labels=labels)`

Matplotlib

Figure & Axes Level Functions

Function / Method	What it does	Example Usage
`plt.subplots()`	Creates a figure and one or more axes objects (recommended OO way)	`fig, ax = plt.subplots()`
`plt.figure()`	Creates a new figure (rarely used alone)	`fig = plt.figure(figsize=(10,6))`
`ax.plot()`, `ax.scatter()`	Same as plt.plot but drawn on specific axes (when using OO style)	`ax.plot(x, y)`
`fig.savefig()`	Saves the figure to a file (png, pdf, jpg, svg, etc.)	`fig.savefig('plot.png', dpi=300)`

Matplotlib

Customization & Labeling

Function / Method	What it does	Example Usage
`plt.title()` / `ax.set_title()`	Sets the main title of the plot	`plt.title("Sales Over Time")`
`plt.xlabel()` / `ax.set_xlabel()`	Sets the label for the x-axis	`plt.xlabel("Days")`
`plt.ylabel()` / `ax.set_ylabel()`	Sets the label for the y-axis	`plt.ylabel("Revenue ($)")`
`plt.legend()` / `ax.legend()`	Displays a legend (needs labels set in plot calls)	`plt.plot(x, y, label="Line 1")` `plt.legend()`
`plt.grid()` / `ax.grid()`	Adds grid lines to the plot	`plt.grid(True)`
`plt.xlim()` / `ax.set_xlim()`	Sets the range/limits for the x-axis	`plt.xlim(0, 10)`
`plt.ylim()` / `ax.set_ylim()`	Sets the range/limits for the y-axis	`plt.ylim(0, 100)`
`plt.xticks()` / `ax.set_xticks()`	Sets custom tick locations and labels on x-axis	`plt.xticks([0, 2, 4], ['Low', 'Mid', 'High'])`
`plt.yticks()` / `ax.set_yticks()`	Same but for y-axis	`ax.set_yticks(range(0, 101, 20))`

Matplotlib

Styling & Visual Properties

Property / Argument	What it does	Common Values
`color=` / `c=`	Sets the color of the line/points/bars	`'blue'`, `'#FF5733'`, `'red'`
`marker=`	Shape of points in scatter or line plots	`'o'`, `'^'`, `'s'`, `'*'`, `'+'`
`linestyle=` / `ls=`	Style of the line	`'-'`, `'--'`, `':'`, `'-.'`
`linewidth=` / `lw=`	Thickness of the line	`1`, `2`, `3.5`
`markersize=` / `ms=`	Size of the markers	`5`, `10`, `15`
`alpha=`	Transparency (0 = fully transparent, 1 = fully opaque)	`0.3`, `0.7`
`label=`	Text to show in legend	`'Sales 2025'`
`edgecolor=`	Border color for bars or markers	`'black'`, `'white'`

Basic Plot Types

Line Plot

```
import matplotlib.pyplot as plt

x = [0, 1, 2, 3, 4, 5]
y = [0, 3, 1, 8, 2, 6]

plt.plot(x, y,
         color='blue',
         marker='o',
         linestyle='-',
         linewidth=2,
         markersize=8)

plt.title("Line Plot Example")
plt.xlabel("Time")
plt.ylabel("Value")
plt.grid(True)
plt.show()
```

Scatter Plot

(used for relationships, distributions, individual points)

```
np.random.seed(42)
x = np.random.rand(50) * 10
y = np.random.rand(50) * 10 + 5

plt.scatter(x, y,
            color='red',
            s=80,              # marker size
            alpha=0.7,         # transparency
            edgecolors='black')

plt.title("Scatter Plot - Random Points")
plt.xlabel("X")
plt.ylabel("Y")
plt.grid(True)
plt.show()
```

Basic Plot Types

Bar Plot

- (used for comparing categories)

```
categories = ['Math', 'Physics', 'Chemistry', 'English']
grades = [88, 75, 92, 80]

plt.bar(categories, grades,
        color=['skyblue', 'lightgreen', 'salmon', 'lightcoral'],
        edgecolor='black')

plt.title("Average Grades by Subject")
plt.xlabel("Subject")
plt.ylabel("Grade")
plt.ylim(0, 100)          # set y-axis range
plt.show()
```

Horizontal Bar Plot

```
plt.barh(categories, grades, color='teal')
plt.title("Horizontal Bar Plot")
plt.xlabel("Grade")
plt.show()
```

Figure & Axes – Understanding the Object Model

The Two Core Objects

- **Figure** (fig): The whole window / canvas / page
- Contains one or more **Axes**
- Controls overall size, saving, background, etc.
- **Axes** (ax): The actual plot area with axes, labels, data
- Where you draw lines, bars, scatter points, etc.
- Most commands you use (plot, set_title, set_xlabel…) go here

Why use fig, ax = plt.subplots()?

- Gives you full control
- Avoids confusion when making multiple plots
- Cleaner for saving, subplots, legends, etc.

Figure & Axes – Understanding the Object Model

```python
import matplotlib.pyplot as plt

# Create figure and axes
fig, ax = plt.subplots(figsize=(8, 5))   # width=8, height=5 inches

# Plot on the axes
ax.plot([1, 2, 3, 4], [10, 40, 20, 60],
        color='purple',
        marker='o',
        linestyle='--',
        label='Trend')

# Customize the axes
ax.set_title("Sales Over Days", fontsize=16)
ax.set_xlabel("Day", fontsize=12)
ax.set_ylabel("Revenue ($)", fontsize=12)
ax.grid(True, linestyle=':', alpha=0.7)
ax.legend()

# Customize the whole figure
fig.suptitle("Monthly Report", fontsize=18)   # big title above everything
fig.tight_layout()                             # auto-adjust spacing

plt.show()
```

```python
# Create figure with 2 rows × 1 column of plots
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(8, 8), sharex=True)

# First subplot
ax1.plot([1,2,3], [5,10,3], 'b-o', label='Line 1')
ax1.set_title("First Plot")
ax1.legend()

# Second subplot
ax2.bar(['A','B','C'], [20,35,15], color='orange')
ax2.set_title("Second Plot")

fig.suptitle("Two Subplots Example", fontsize=16)
fig.tight_layout()
plt.show()
```

Figure & Axes – Understanding the Object Model

Quick Reference

```
ax.plot()          → draw line
ax.scatter()       → draw points
ax.bar()           → vertical bars
ax.set_title()     → set plot title
ax.set_xlabel()    → x-axis label
ax.set_ylabel()    → y-axis label
ax.legend()        → show legend
ax.grid()          → add grid lines
ax.set_xlim()      → set x-axis limits
ax.set_ylim()      → set y-axis limits
```